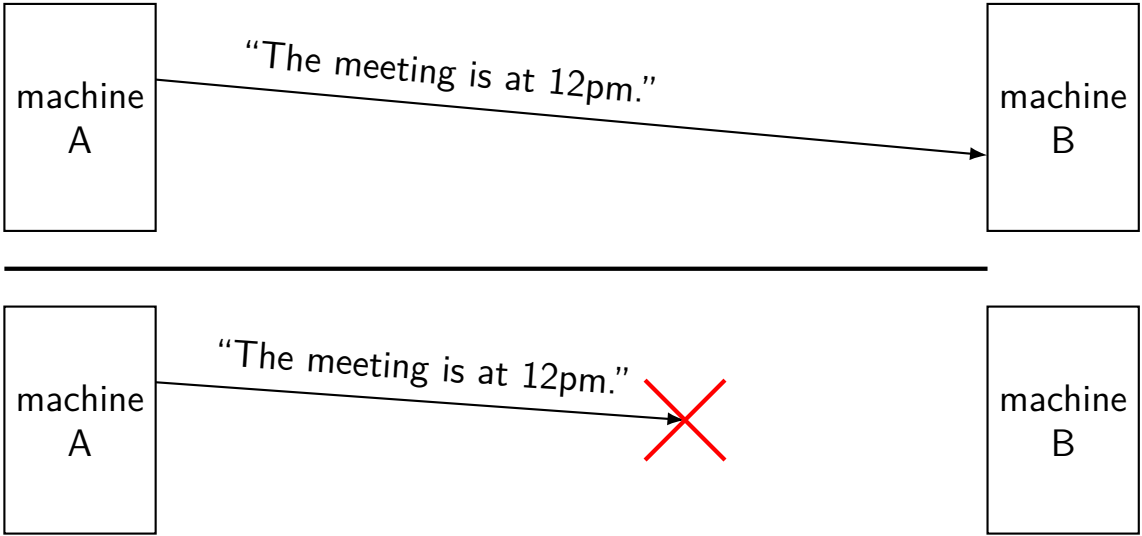
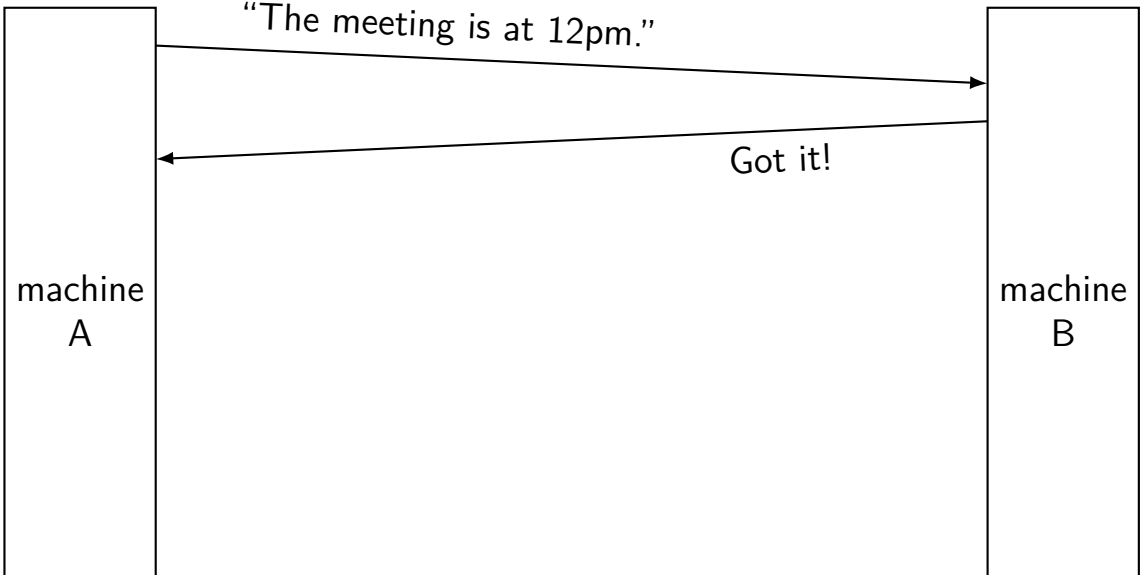
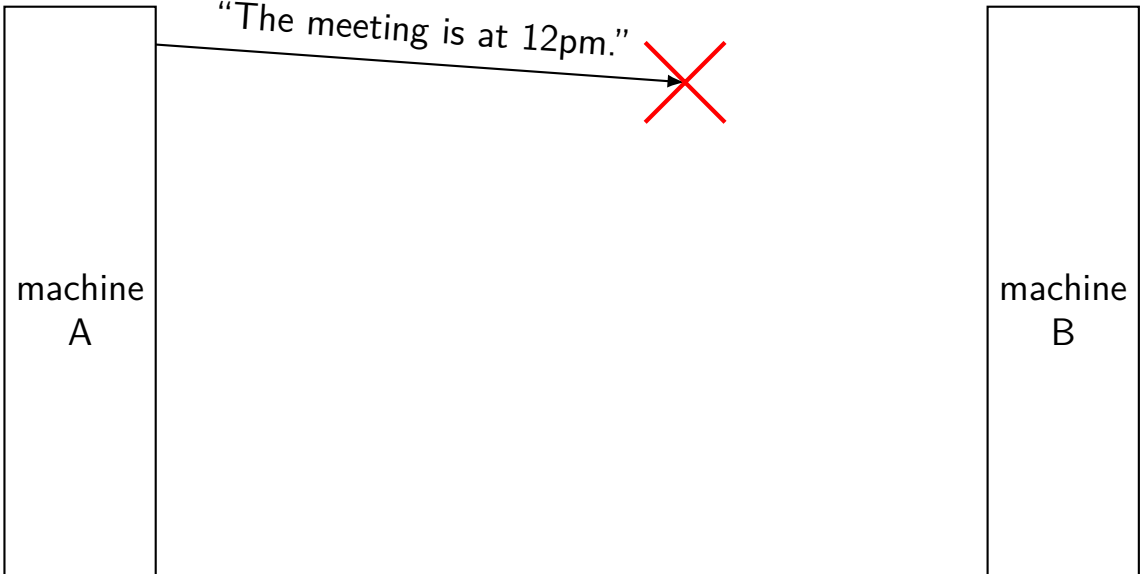
dealing with network message lost



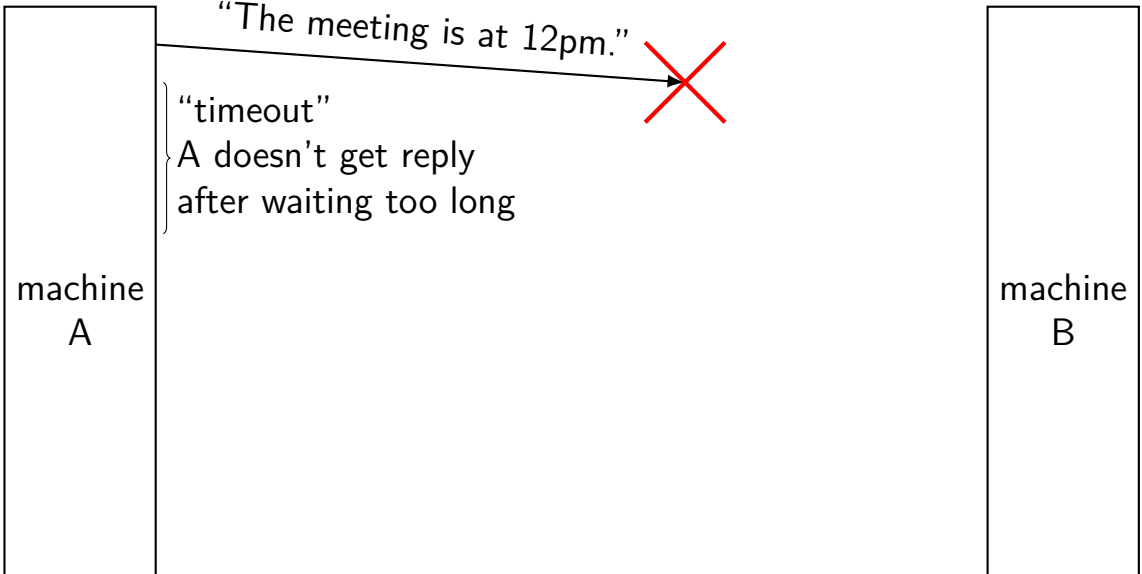
handling lost message: acknowledgements



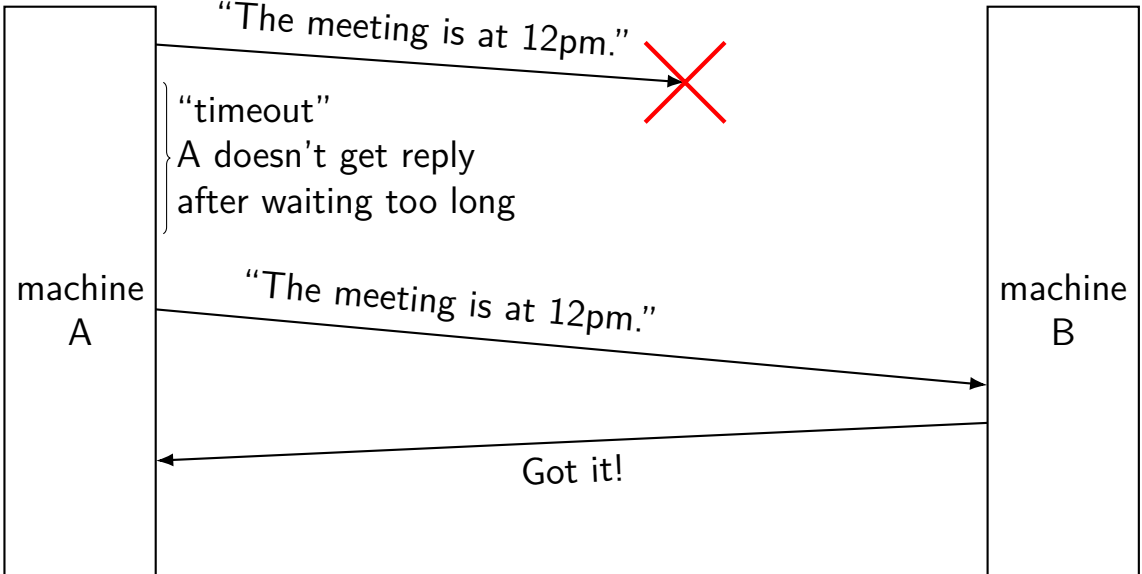
handling lost message



handling lost message



handling lost message



protocol so far

on sender: until ACK received:

- (re)send frame of data
- wait fixed amount of time for ACK

on receiver: continuously:

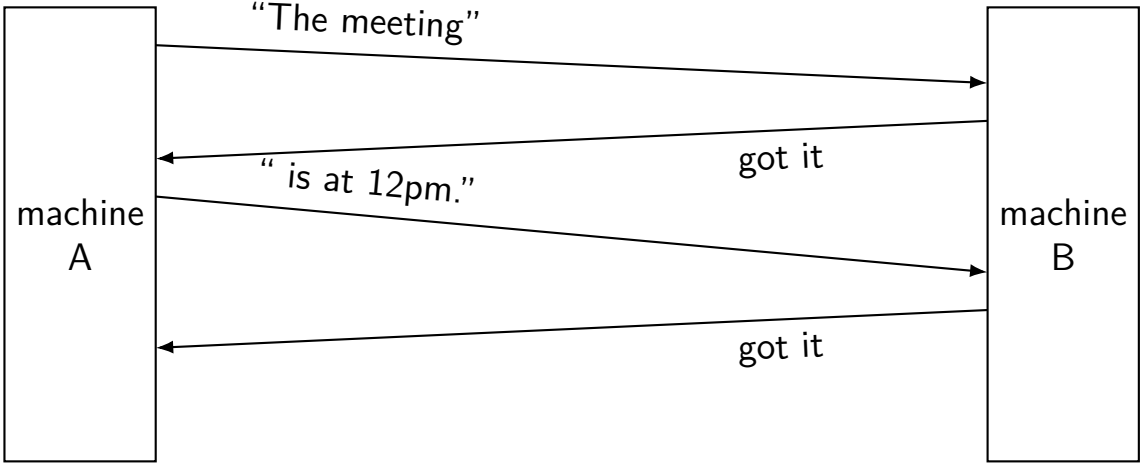
- wait for frame of data
- send ACK back

problem

really want to send multiple frames

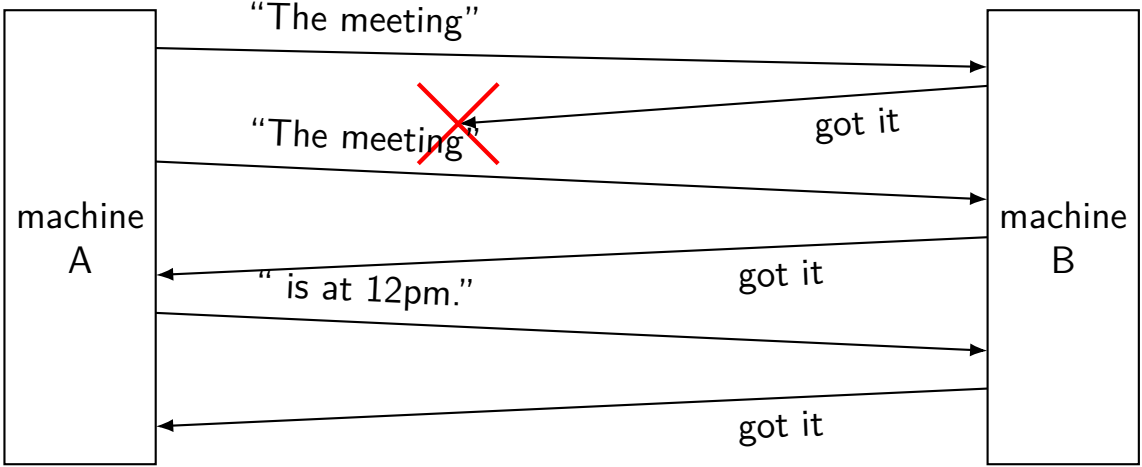
example: data split in multiple pieces

splitting messages: try 1

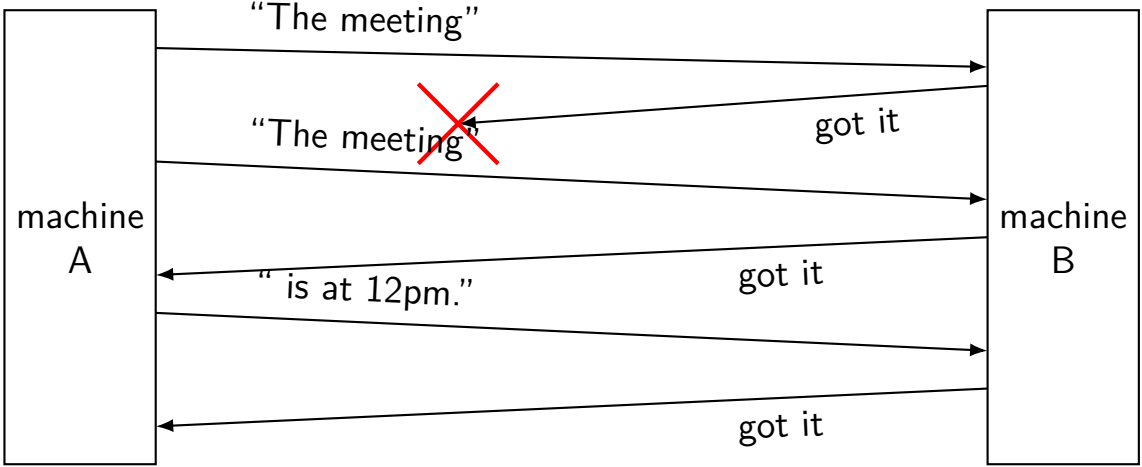


reconstructed message:
The meeting is at 12pm.

splitting messages: try 1 — problem 1



splitting messages: try 1 — problem 1



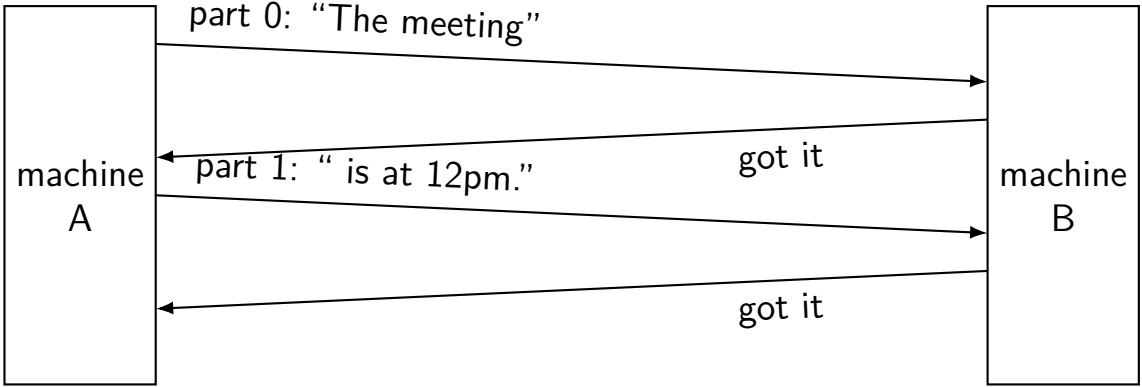
reconstructed message:
The meetingThe meeting is at 12pm.

exercise: other problems?

other scenarios where we'd also have problems?

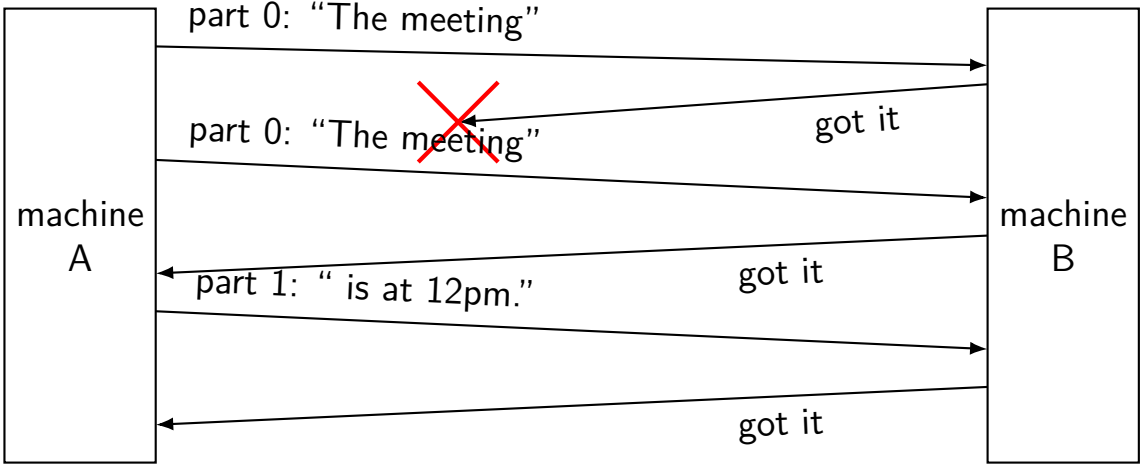
1. message (instead of acknowledgment) is lost
2. first message from machine A is delayed a long time by network
3. acknowledgment of second message lost instead of first

splitting messages: try 2



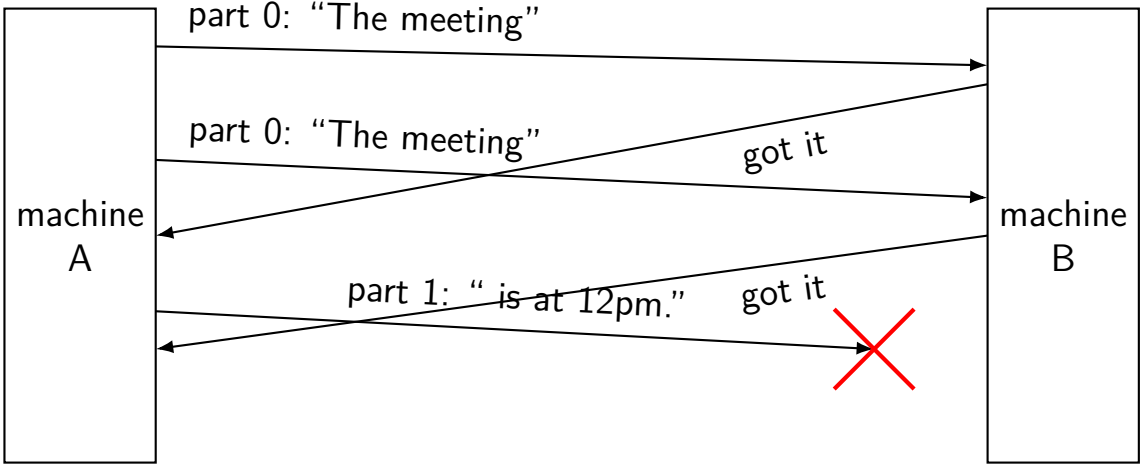
reconstructed message:
The meeting is at 12pm.

splitting messages: try 2 — missed ack



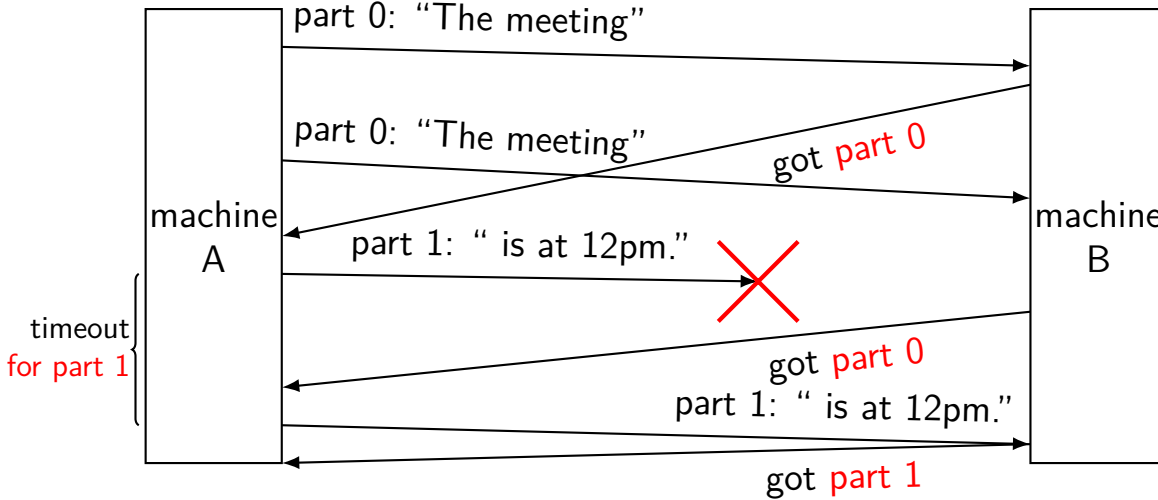
reconstructed message:
The meeting is at 12pm.

splitting messages: try 2 — problem



A thinks: part 1 + part 2 acknowleged!

splitting messages: version 3



sequence numbers

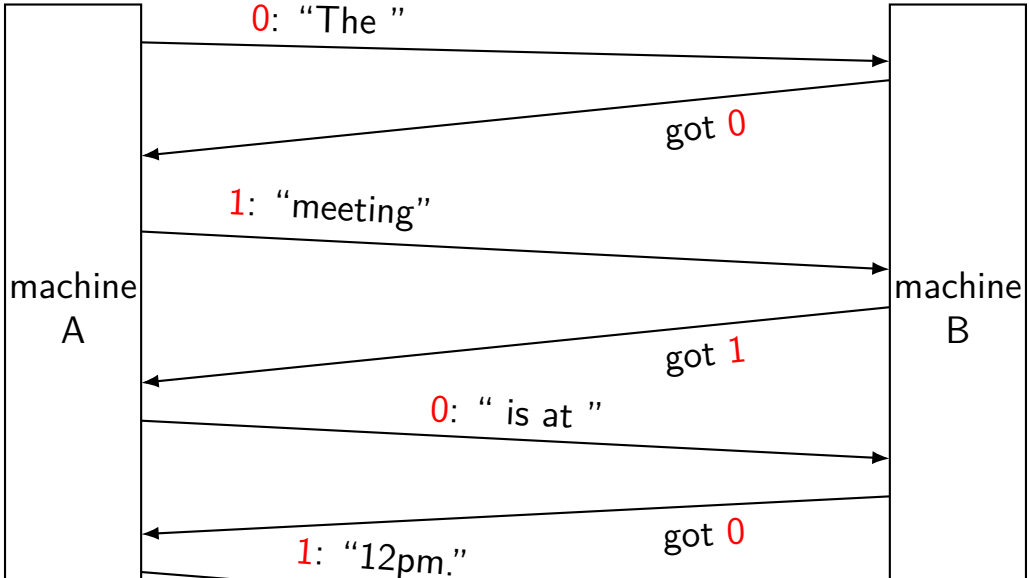
call the ‘part’ label *sequence number*

important question: how large can they get?

if we never reuse them — infinite!

so *really* want to reuse them

1-bit sequence number



‘stop and wait’

machine A is only sending one thing at a time

never start sending next thing until after sending previous thing

stop-and-wait exercise (receive, 1)

machine B receives `0: X`

machine B sends `got 0`

machine B receives `0: X`

what should machine B do now?

A. send `got 0` B. send `got 1` C. send nothing

stop-and-wait exercise (receive, 2)

machine B receives `0: X`

machine B sends `got 0`

machine B receives `1: X`

what should machine B do now?

A. send `got 0` B. send `got 1` C. send nothing

stop-and-wait exercise (receive, 3)

machine B receives `0: X`

machine B sends `got 0`

machine B receives `1: Y`

machine B sends `got 1`

machine B receives `0: X`

what should machine B do now?

A. send `got 0` B. send `got 1` C. send nothing

stop-and-wait exercise (send, 1)

A trying to send 'X', then 'Y', then 'Z'

machine A sends `0: X`

machine A sends `0: X`

machine A receives `got 0`

machine A sends `1: Y`

machine A receives `got 0`

what should machine A do now?

A. send `0: X` again B. send `1: Y` again

C. send `0: Z` D. something else

stop-and-wait exercise (send, 2)

A trying to send ‘X’, then ‘Y’, then ‘Z’

machine A sends `0: X`

machine A sends `0: X`

machine A receives `got 0`

machine A sends `1: Y`

machine A receives `got 1`

what should machine A do now?

A. send `0: X` again B. send `1: Y` again

C. send `0: Z` D. something else

bandwidth / throughput

bandwidth / data rate: maximum rate we can send per unit time

- most commonly measuring the speed of a link

1 gigabit/second = transmit 1 bit / nanosecond

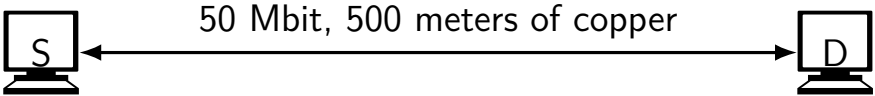
throughput: acheived rate per unit time

- often lower than total bandwidth because of losses
- (we'll give several examples throughout the semester)

latency (1)

latency: time for message: SOURCE $\rightarrow$ DEST

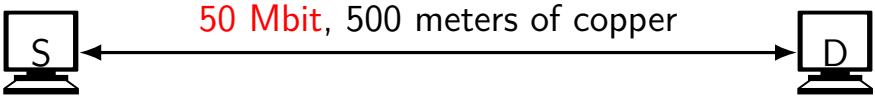
example: 1000 bit message from S to D:



latency (1)

latency: time for message: SOURCE $\rightarrow$ DEST

example: 1000 bit message from S to D:



one bit sent each $1/50\text{M}$ second $= 0.02\ \mu$s

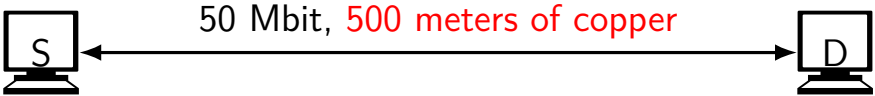
1000 bits take $0.02 \times 1000 = 20\ \mu$s to sent

"transmission delay"

latency (1)

latency: time for message: SOURCE $\rightarrow$ DEST

example: 1000 bit message from S to D:



one bit sent each 1/50M second $= 0.02$ μs

1000 bits take $0.02 \times 1000 = 20$ μs to sent

"transmission delay"

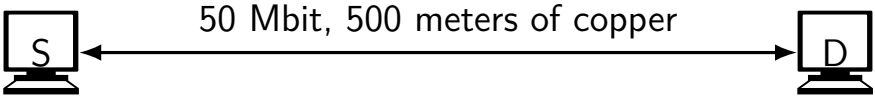
$+ 2.2$ microseconds for bit to go down cable (2.3×10^8 m/s)

"propogation delay"

latency (1)

latency: time for message: SOURCE $\rightarrow$ DEST

example: 1000 bit message from S to D:



one bit sent each 1/50M second $= 0.02$ μs

1000 bits take $0.02 \times 1000 = 20$ μs to sent

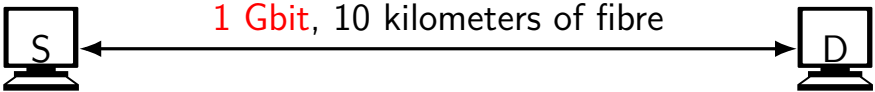
"transmission delay"

$+ 2.2$ microseconds for bit to go down cable (2.3×10^8 m/s)

"propogation delay"

total latency of about 22.2 μs

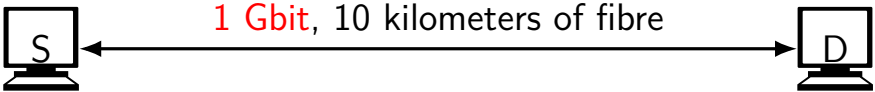
latency (1, ex)



exercise: latency for 20000 bit message from S to D

assume speed of signal through fiber of 2.0×10^8 m/s

latency (1, ex)



exercise: latency for 20000 bit message from S to D

assume speed of signal through fiber of 2.0×10^8 m/s

latency (1, ex)



exercise: latency for 20000 bit message from S to D

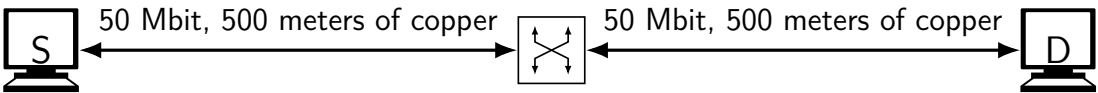
assume speed of signal through fiber of 2.0×10^8 m/s

latency (2)

example: 1000 bit packet from S to D

assume when message is received:

- 5 other 1000-bit packets in queue; no extra bits between packets
- no other switch processing time



latency from S to switch: 22.2 μs

time waiting on switch: wait $20 \times 5 = 100$ μs for 5 other packets

- "queueing delay"

latency from switch to D: 22.2 μs

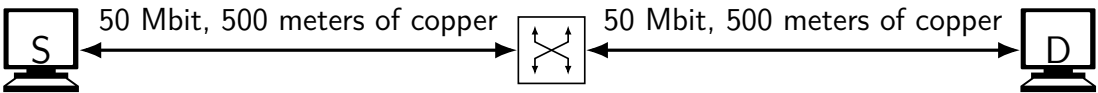
latency (2)

example: 1000 bit packet from S to D

assume when message is received:

5 other 1000-bit packets in queue; no extra bits between packets

no other switch processing time



latency from S to switch: 22.2 μs

time waiting on switch: wait $20 \times 5 = 100$ μs for 5 other packets

"queueing delay"

latency from switch to D: 22.2 μs

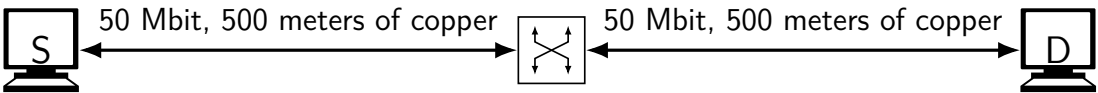
latency (2)

example: 1000 bit packet from S to D

assume when message is received:

5 other 1000-bit packets in queue; no extra bits between packets

no other switch processing time



latency from S to switch: 22.2 μs

time waiting on switch: wait $20 \times 5 = 100$ μs for 5 other packets

"queueing delay"

latency from switch to D: 22.2 μs

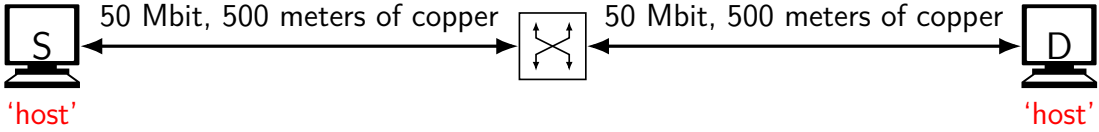
latency (2)

example: 1000 bit packet from S to D

assume when message is received:

5 other 1000-bit packets in queue; no extra bits between packets

no other switch processing time



latency from S to switch: 22.2 μs

time waiting on switch: wait $20 \times 5 = 100$ μs for 5 other packets

"queueing delay"

latency from switch to D: 22.2 μs

round trip time

round-trip-time (RTT): time for message: SOURCE $\rightarrow$ DEST $\rightarrow$ SOURCE

jitter

variation in latency

most commonly from changing queuing delays

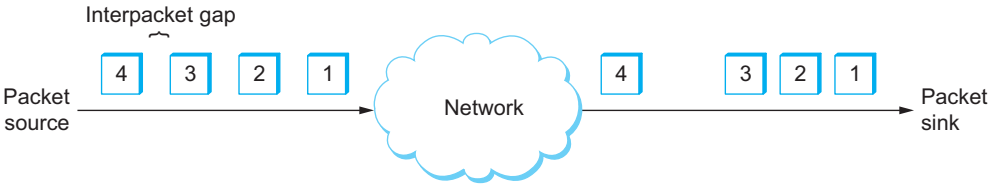


Figure 20 from Section 1.5 of *Computer Networks: A Systems Approach* (6th ed) (Peterson and Davie)

stop-and-wait performance

stop-and-wait protocol

assuming no packets lost/corrupted

about one packet per round-trip time

example: local ethernet

my home wired network: 0.6 ms round trip time

typical packet has about 1400 bytes $=$ 11200 bits of data

throughput with stop-and-wait:
$11200\text{b}/0.6\text{ms} \approx 19000\text{b}/\text{ms} = 19\ 000\ 000\text{b}/\text{s} = 19\text{Mbit}/\text{s}$

available bandwidth is about 1 Gbit/s

example: local ethernet

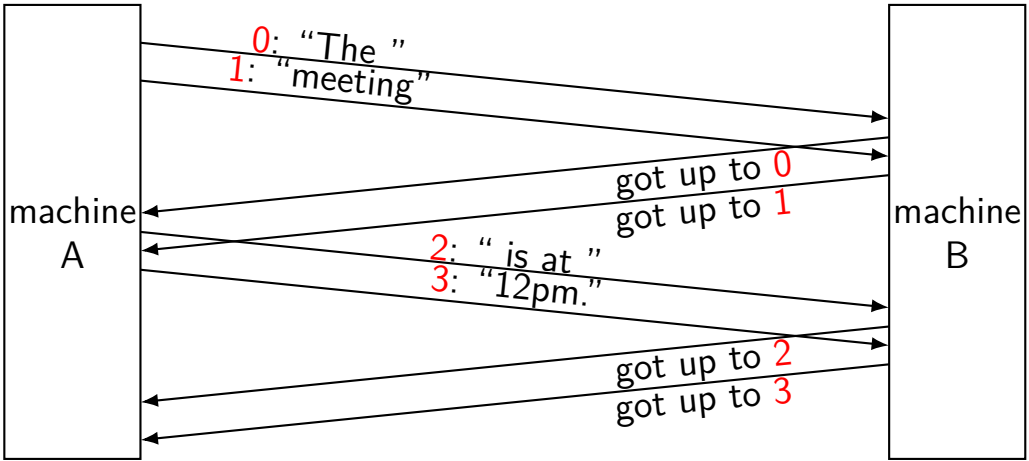
my home wired network: 0.6 ms round trip time

typical packet has about 1400 bytes = 11200 bits of data

throughput with stop-and-wait:
$11200\text{b}/0.6\text{ms} \approx 19000\text{b}/\text{ms} = 19\ 000\ 000\text{b}/\text{s} = 19\text{Mbit}/\text{s}$

available bandwidth is about 1 Gbit/s

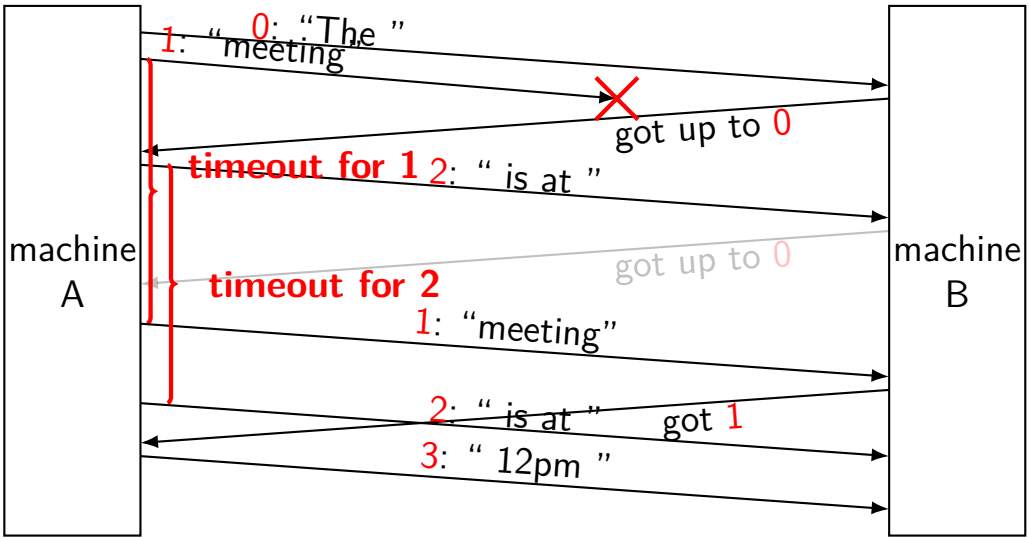
sending two at a time



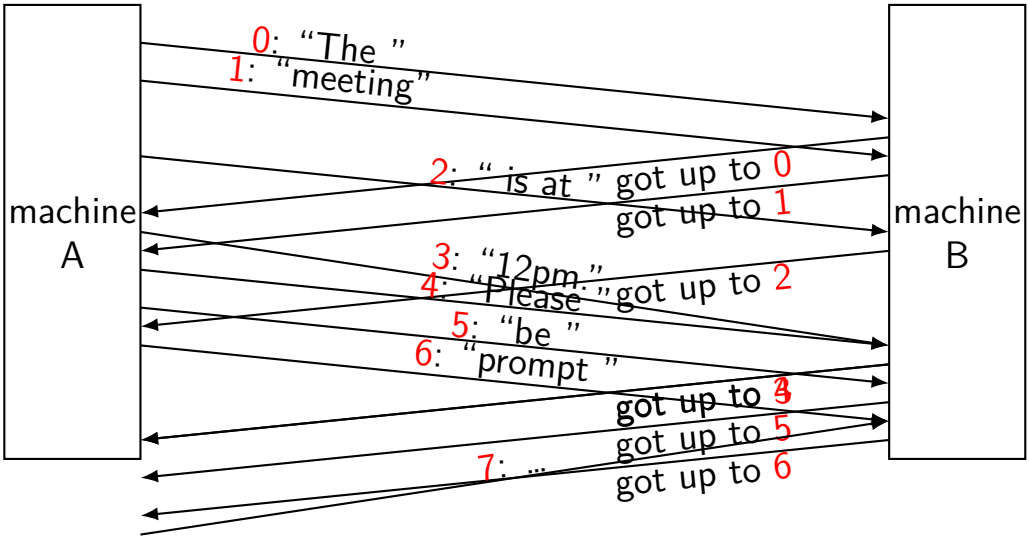
key idea: always have two in flight

send next when previous ack'd

timeouts per message

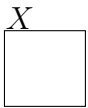


sending three at a time



choose "window size" to have in flight

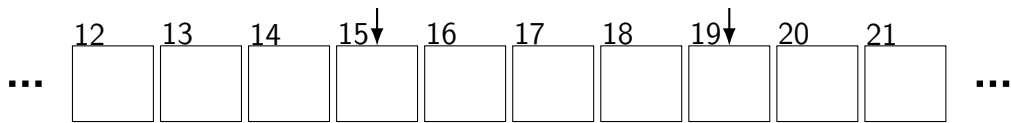
sender window tracking



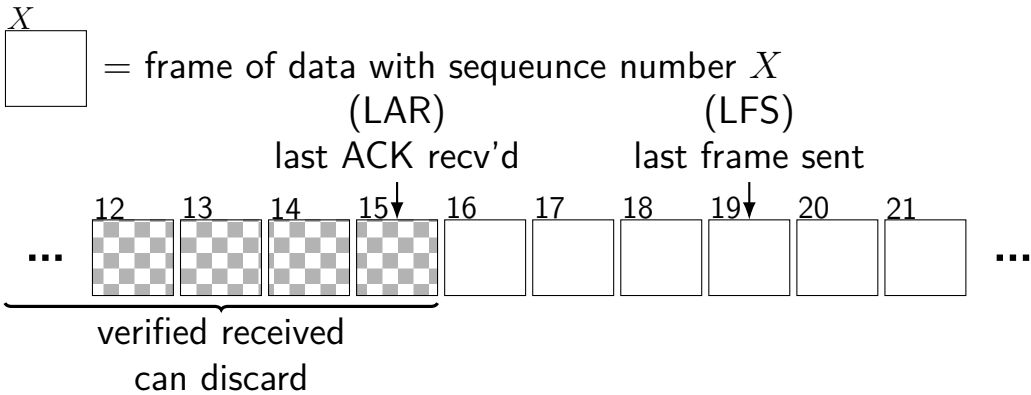
$=$ frame of data with sequeunce number X

(LAR)
last ACK recv'd

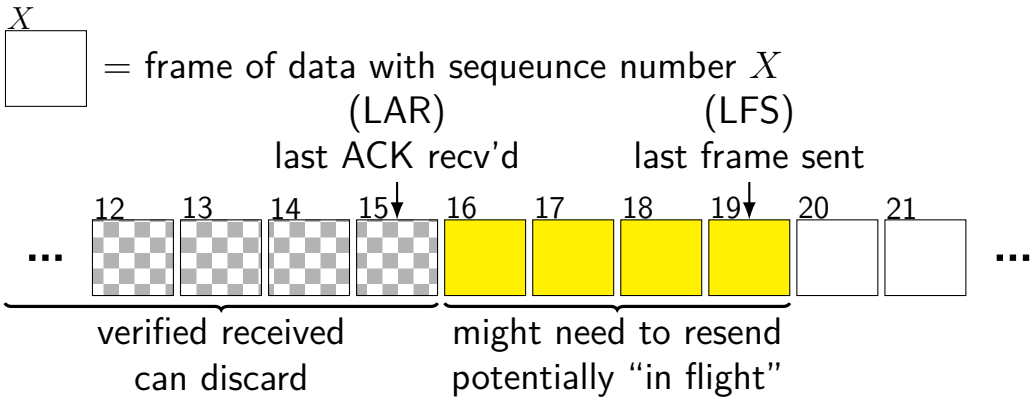
(LFS)
last frame sent



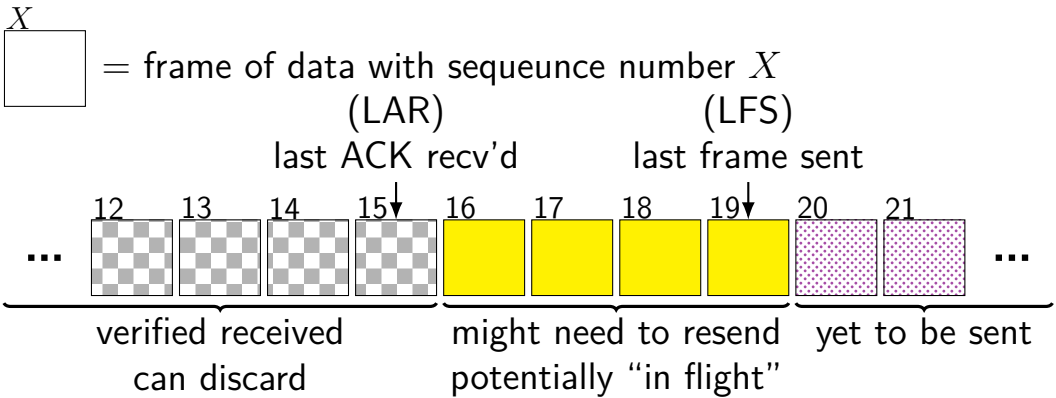
sender window tracking



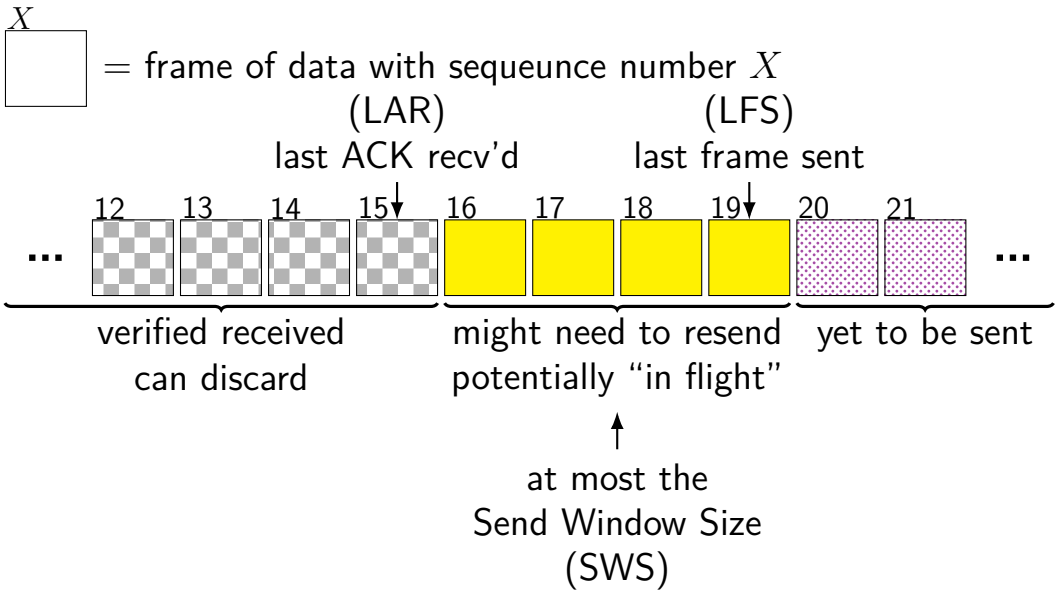
sender window tracking



sender window tracking



sender window tracking



exercise 1: out-of-bounds ACK

last ACK recv'd (LAR)	10
last frame sent (LFS)	15
send window size (SWS)	5

what probably happened if we receive an ACK for...

9? 10? 13? 16?

A. only possible if network reorders frames

B. only possible from undetected frame corruption

C. lost ACK for frame ≤ 10

D. lost ACK for frame > 10

E. lost frame 11

F. resent frame from timeout

exercise 2: sender logic

last ACK recv'd (LAR)	10
last frame sent (LFS)	15
send window size (SWS)	5

In this case, there's a timeout that will trigger frame 13 to be resent. If still active, this timeout should be cancelled upon ...

A. receiving ACK 12 B. receiving ACK 13

C. receiving ACK 14 D. sending frame 16

exercise 3a: new data

last ACK recv'd (LAR)	4
last frame sent (LFS)	6
send window size (SWS)	5

if we compute a new frame of data with sequence number 7 to eventually send, we should

A. send it now, advancing LFS
B. wait until we get an ACK for 5 or 6 to send it
C. wait until we get an ACK for 6 to send it
D. wait until the frame with sequence number 6 is resent to send it D. so

exercise 3b: new data

last ACK recv'd (LAR)	4
last frame sent (LFS)	8
send window size (SWS)	4

if we compute a new frame of data with sequence number 9 to eventually send, we should

A. send it now, advancing LFS
B. wait until we get an ACK for 5 or 6 to send it
C. wait until we get an ACK for 6 to send it
D. decline to accept the data because we will never be able to send it
E. something else

sender logic summarized

track variables:

- LFS (last frame sent)
- LAS (last ACK recv'd)
- SWS (send window size)

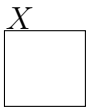
when receiving ACK $LAR < X \leq LFS$:

- LAR $\leftarrow X$
- clear any timers to resend frames $\leq X$

whenever SWS [send window size] $>$ LFS - LAR *and* data for frame LFS $+ 1$ is available:

- send frame LFS $+ 1$
- set timer to resend frame LFS $+ 1$
- LFS $\leftarrow$ LFS $+ 1$

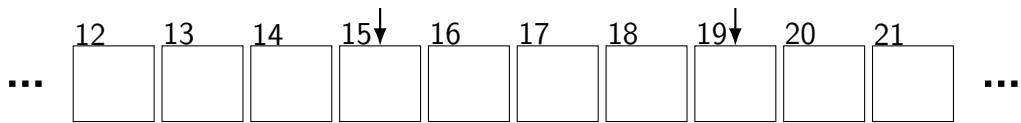
receiver window tracking



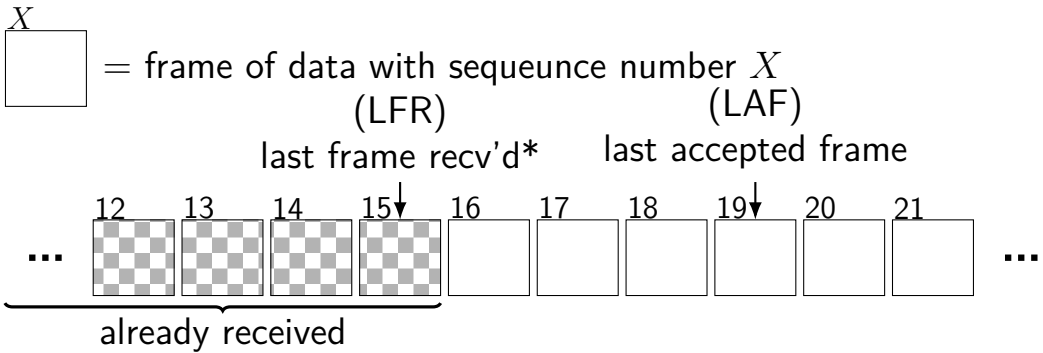
= frame of data with sequeunce number X

(LFR)
last frame recv'd*

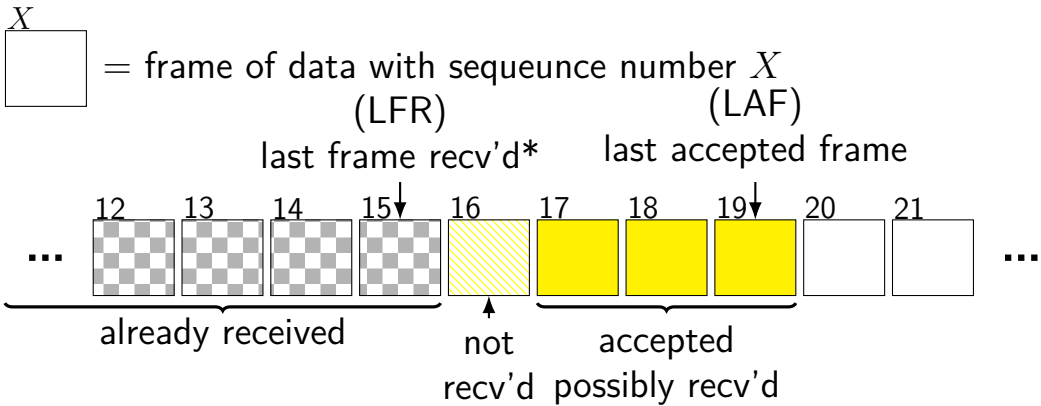
(LAF)
last accepted frame



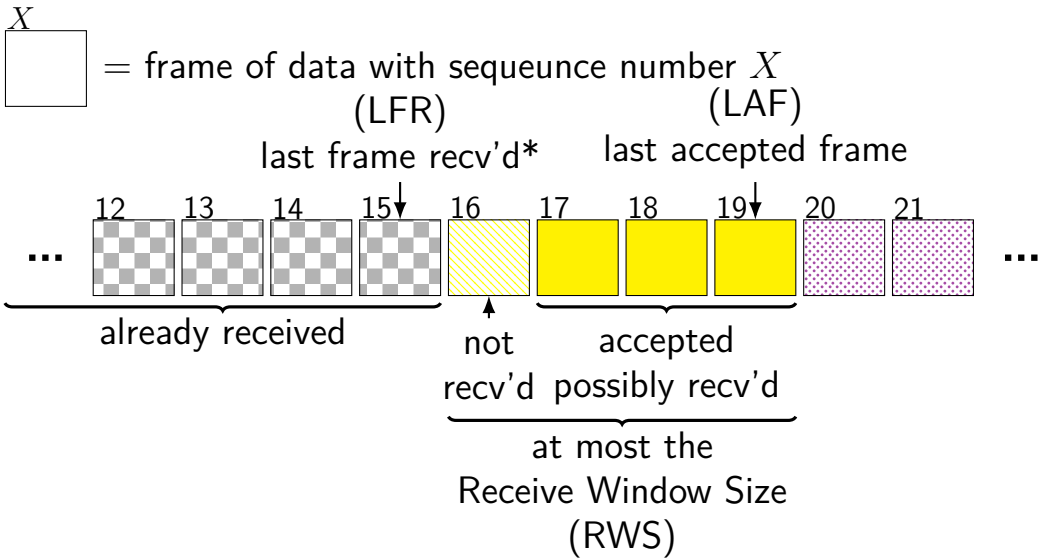
receiver window tracking



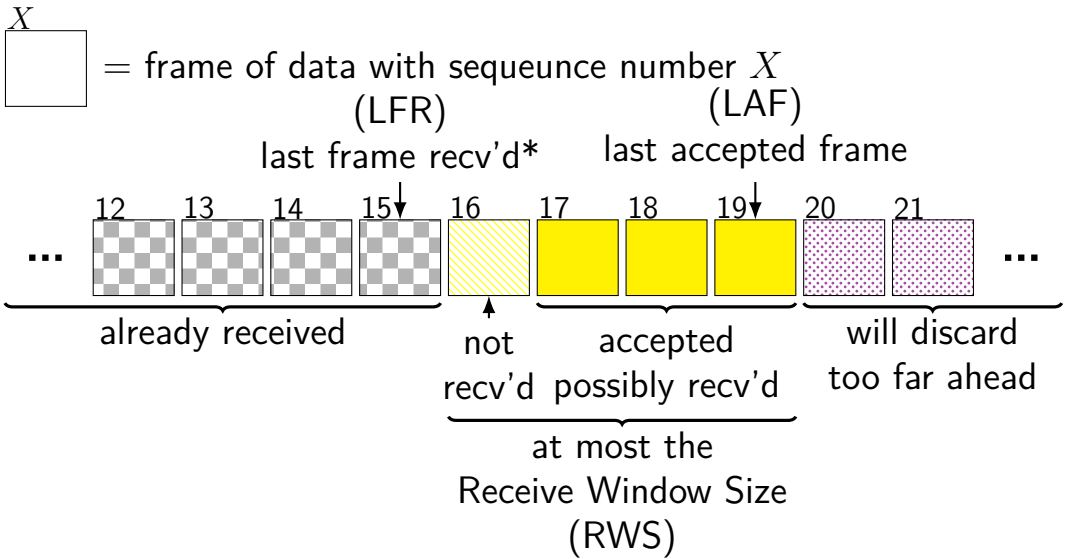
receiver window tracking



receiver window tracking



receiver window tracking



receiver logic summarized

track variables:

- LFR (last frame recv'd) — excludes frames after a missing frame
- LAS (last accepted frame)
- RWS (receive window size)

when receiving frame $LFR < X \leq LAS$:

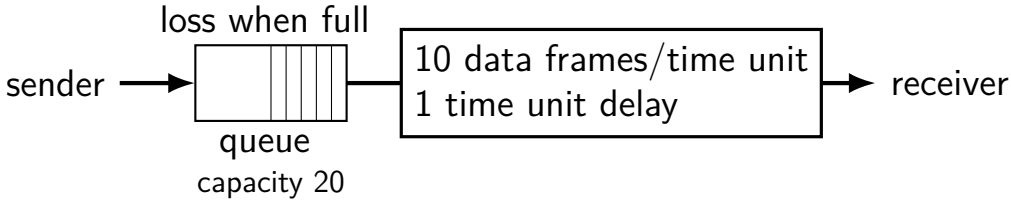
LFR $\leftarrow$ (first missing frame after LFR) - 1

- only advances if X = LFR
- could advance by more than one if frames previously out of order

LAS $\leftarrow$ LFR + RWS

- only advances if X = LFR

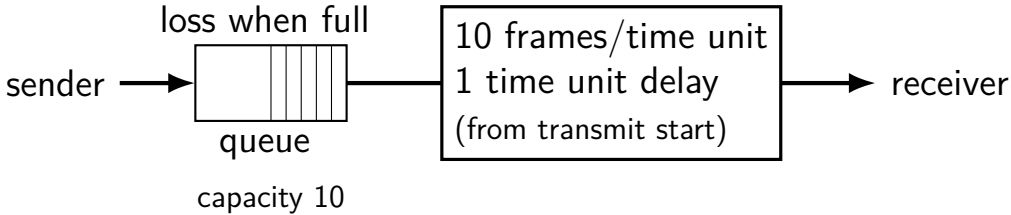
simple network model



simulator from upcoming assignment

command line `--delay 1 --bandwidth-forward 10 --bandwidth-backward 100 --buffer 30`

exercise: forward latency

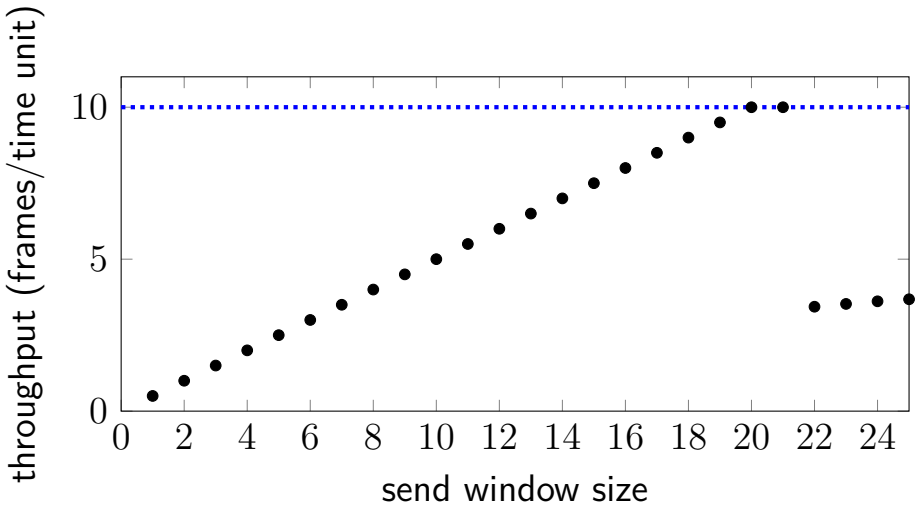


minimum latency = 1 time unit

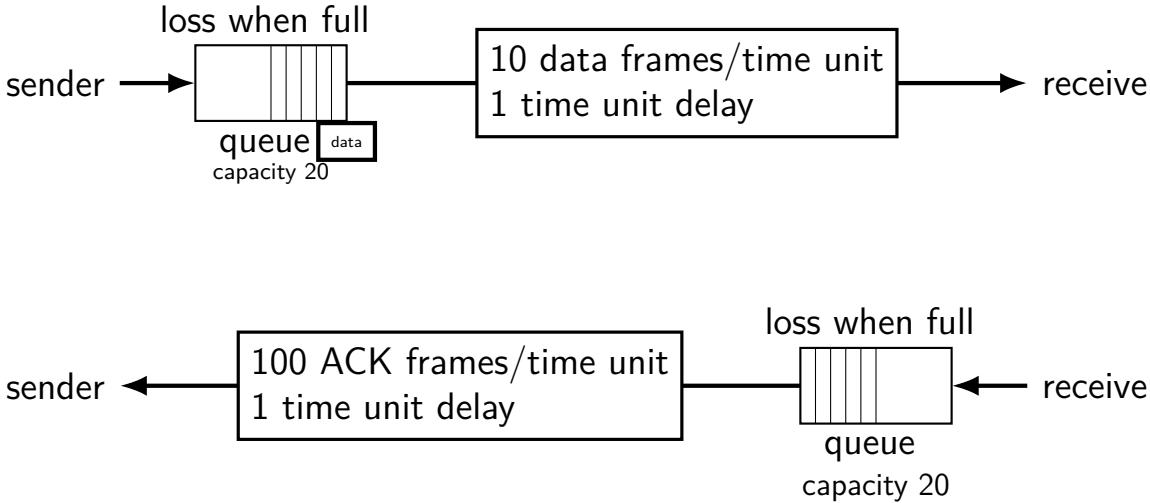
exercise: maximum latency?

A. 1 time unit B. 1.1 time unit C. 1.2 time unit
C. 1.4 time unit D. 1.9 time unit E. 2.0 time unit
F. 2.1 time unit G. something else

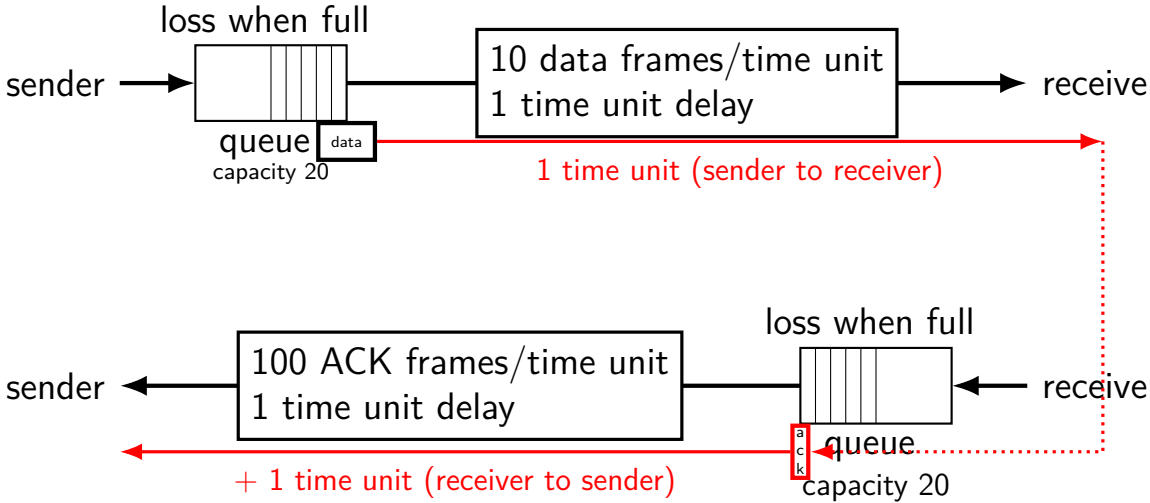
throughput and window size



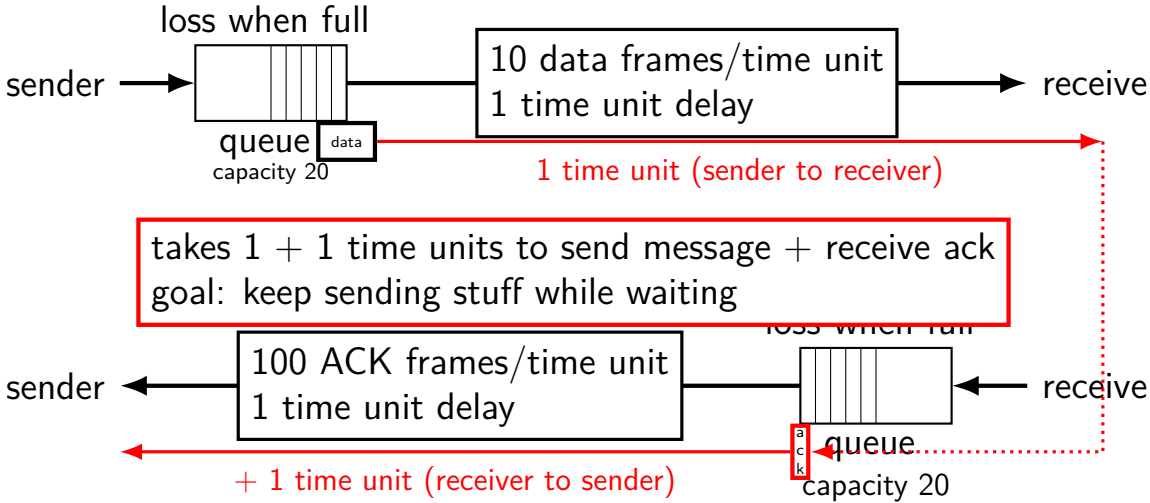
packet transit time



packet transit time



packet transit time



filling the pipe

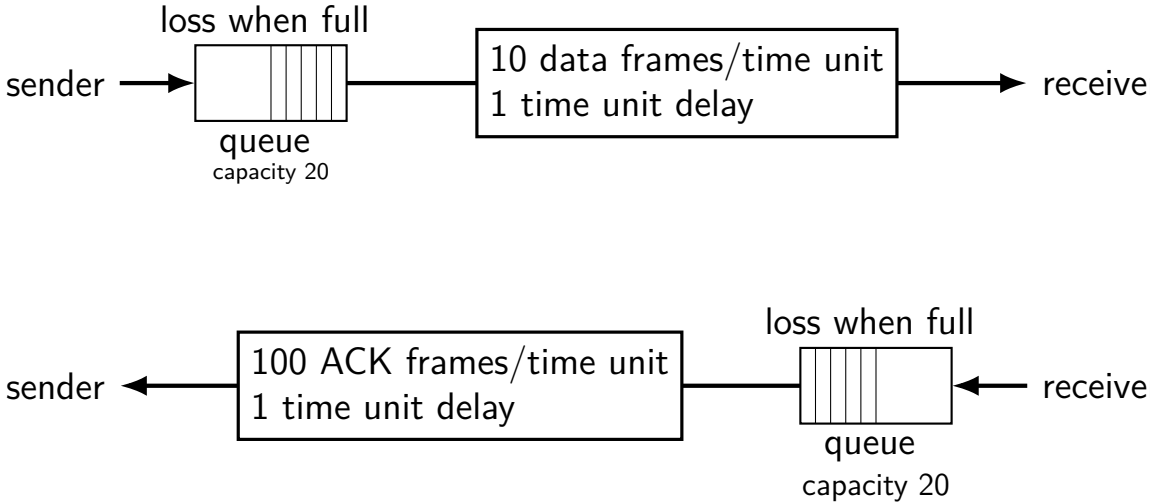
round-trip time of 2 time units

from send data to receive ACK (assuming no queuing delay)

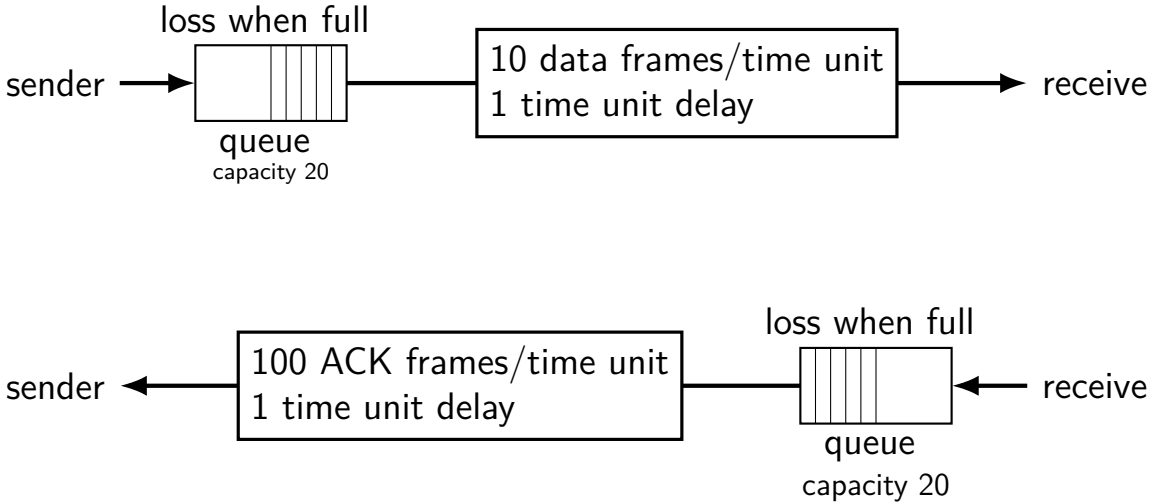
can send 10 data frames per time unit

= can send 20 data frames while waiting for ACK

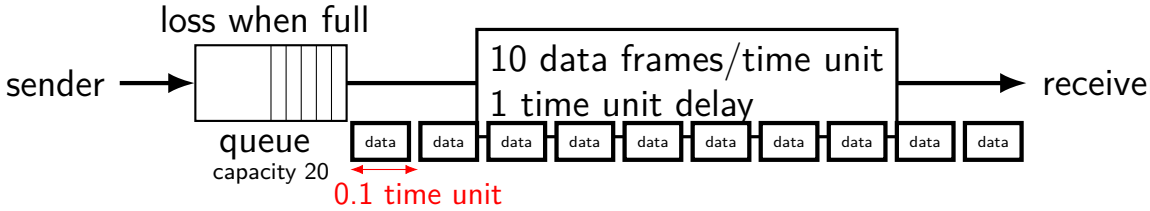
filling the pipe



filling the pipe



filling the pipe



sliding windows solve...

reliability on unrelaible link

congestion control

keep network from being overloaded
...by having window sizes set correctly

flow control

keep sender from getting too far ahead of sender
...by having window sizes set correctly

sequence number wraparound

protocol so far requires arbitrarily large sequence numbers

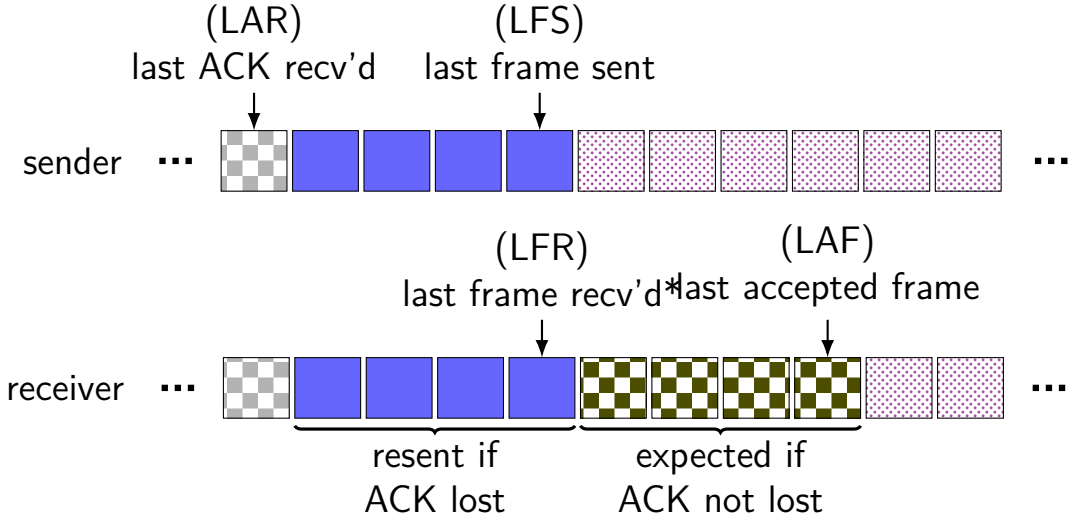
- doing $<$ and $>$ checks on sequence number, so they need to increase

would like to use smaller sequence numbers

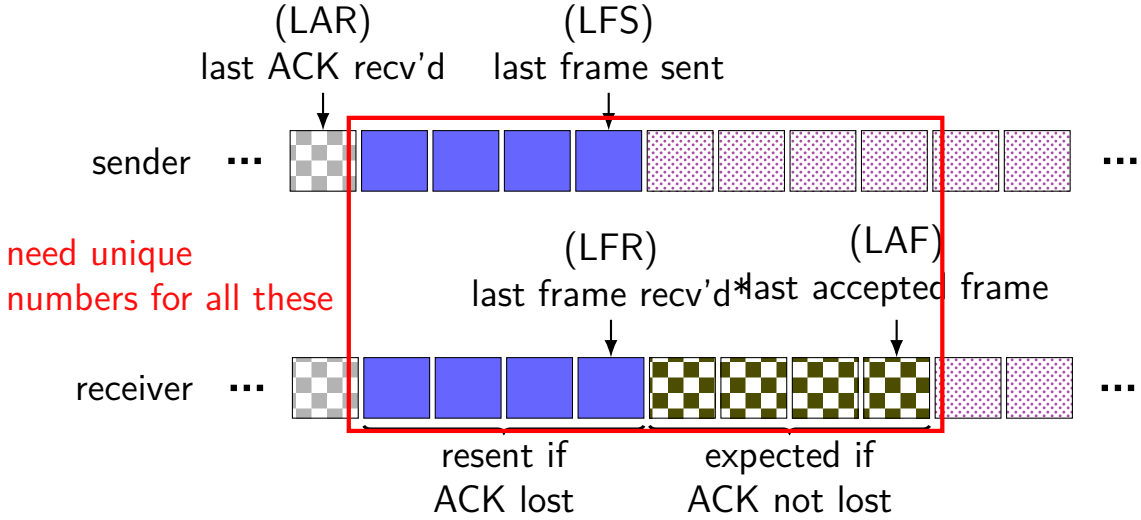
- think: transferring multi-gigabyte file

question: what goes wrong when we reuse sequeunce numbers?

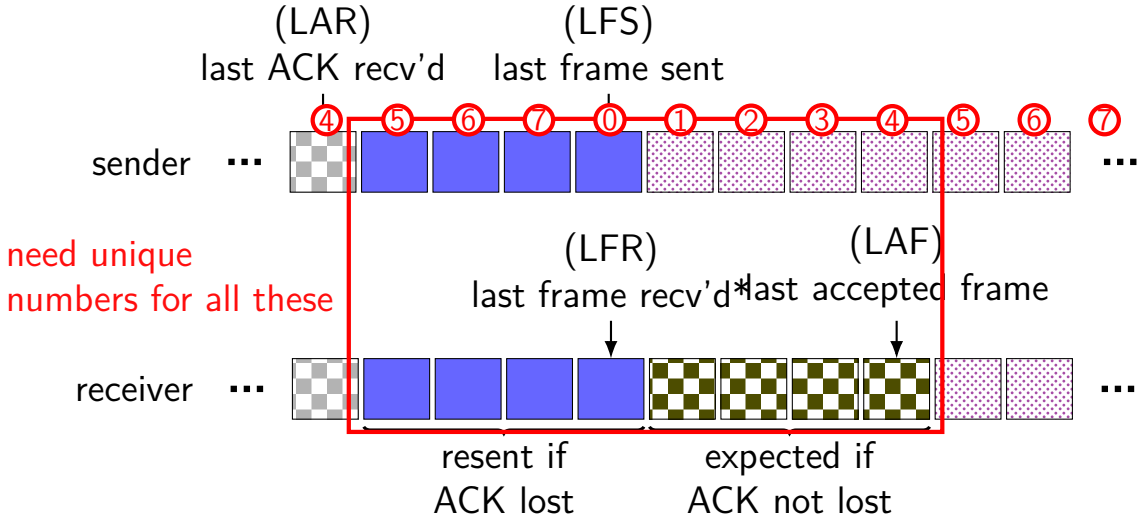
sender/receiver desync: missing ACKs



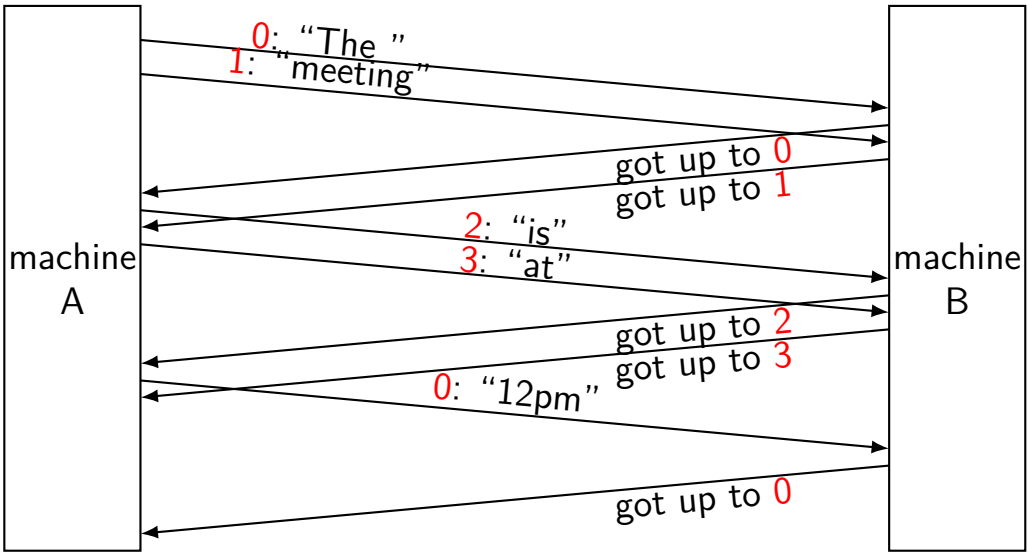
sender/receiver desync: missing ACKs



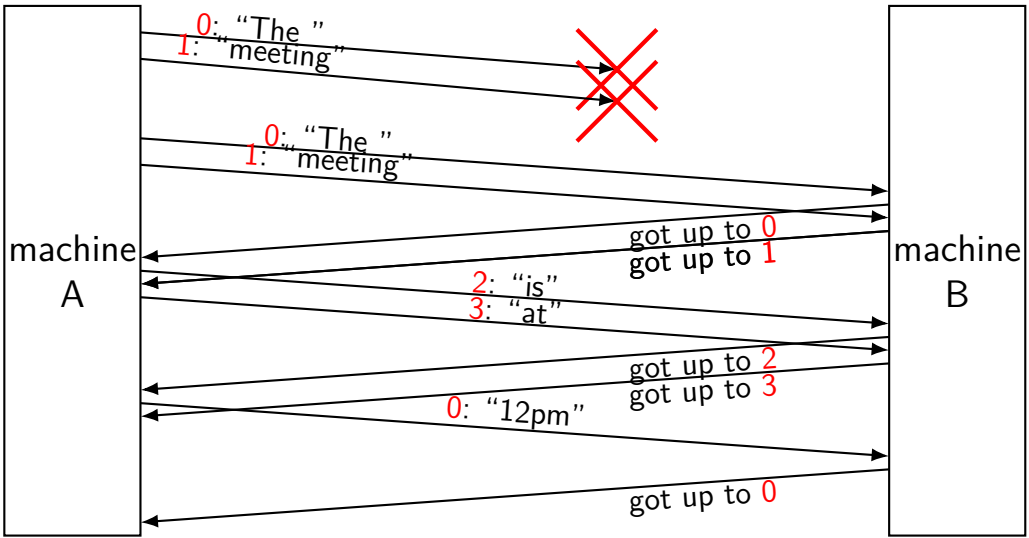
sender/receiver desync: missing ACKs



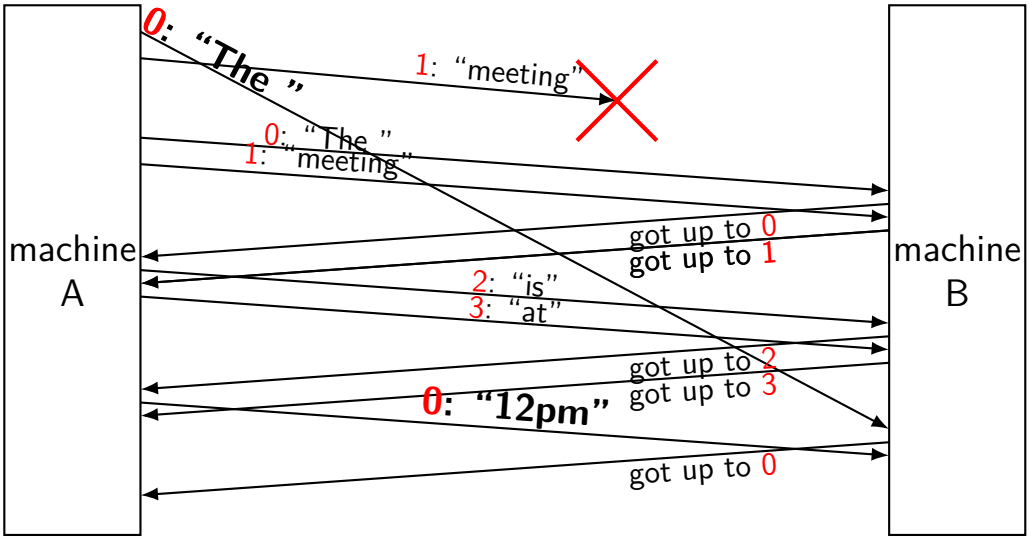
wraparound


machine A
machine B
0: "The "
1: "meeting"
got up to 0
got up to 1
2: "is"
3: "at"
got up to 2
got up to 3
0: "12pm"
got up to 0

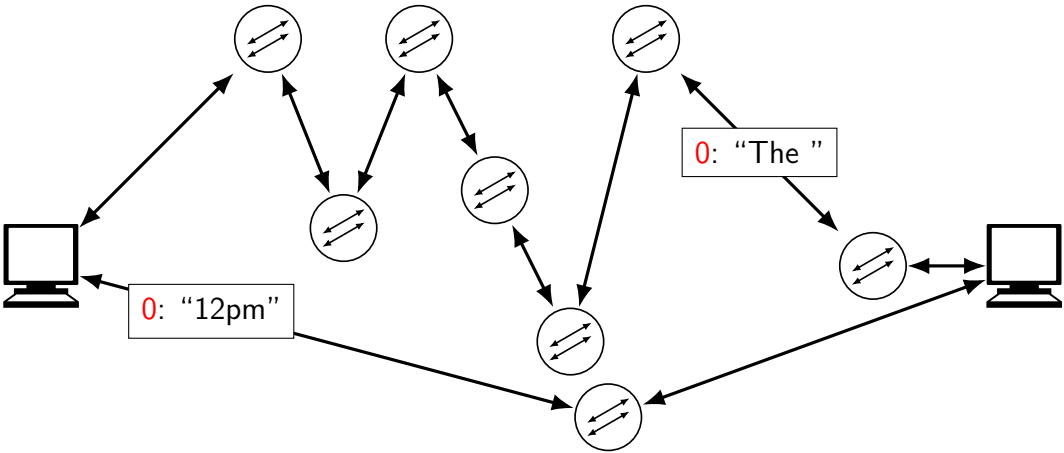
loss and resend?



very bad reordering



possible reason



sequence numbers in practice

TCP tries to assume 120 second "maximum segment lifetime"

segment = TCP's name for a packet

original TCP used 32-bit sequence number identifying *byte* number (not segment number)

problem: means wraparound happens on modern (Gigibit+) links in seconds!

sequence numbers in practice

TCP tries to assume 120 second "maximum segment lifetime"

- segment = TCP's name for a packet

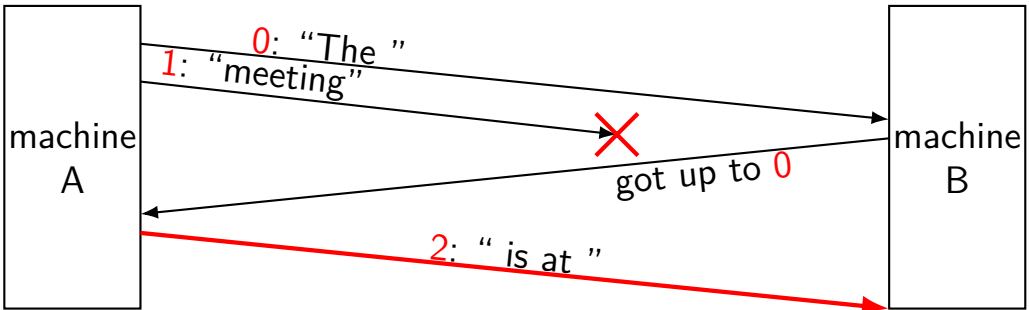
original TCP used 32-bit sequence number identifying *byte* number (not segment number)

problem: means wraparound happens on modern (Gigibit+) links in seconds!

workaround: add *additional* 32-bit timestamp field

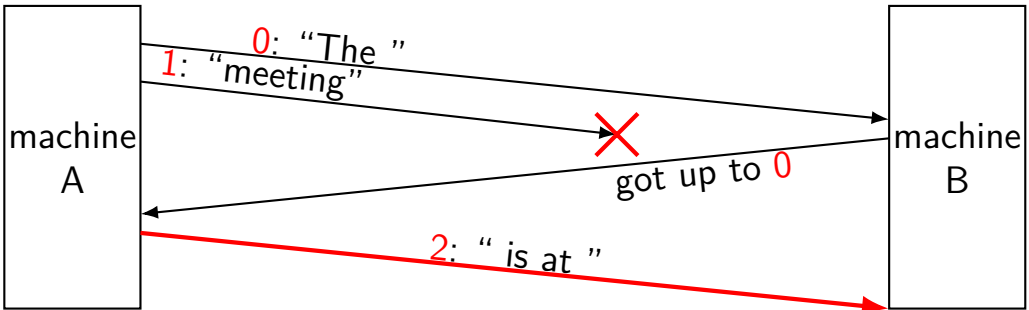
- used to detect/discard duplicates
- can also be used to set timeouts and/or window sizes

missing messages?



question: what should receiver do with sequence number 2?

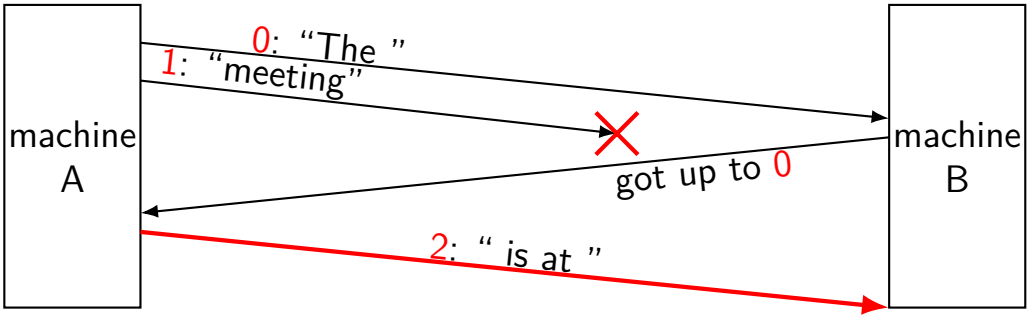
missing messages?



question: what should receiver do with sequence number 2?

one idea: ignore it?

missing messages?

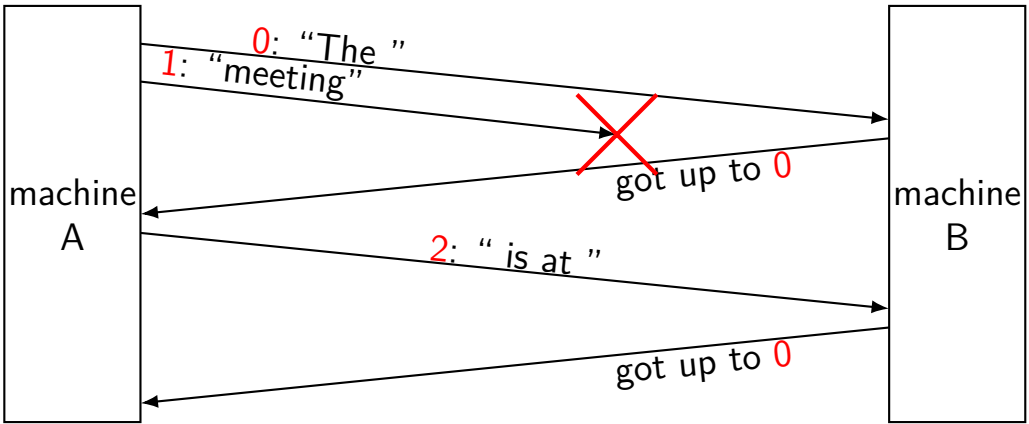


question: what should receiver do with sequence number 2?

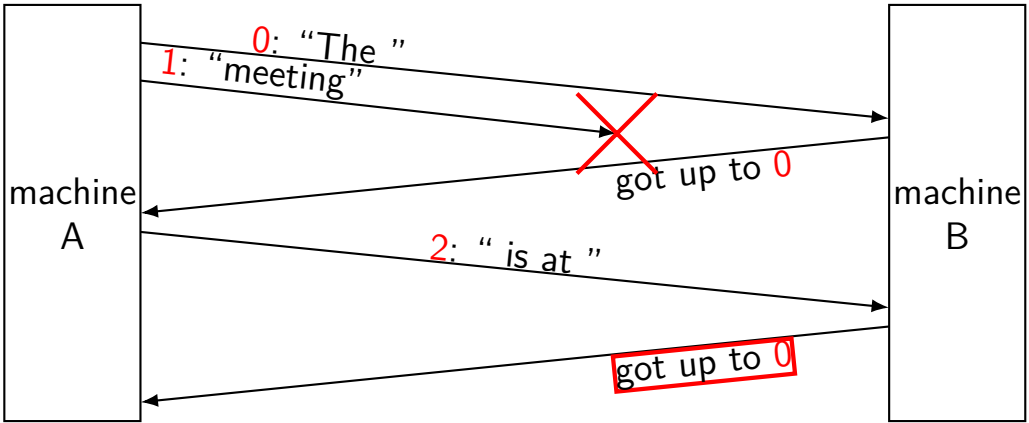
one idea: ignore it?

better idea: send something back to sender

better idea: always ACK



better idea: always ACK



only ACK x if everything up to and including x received

intuition: ACK tells sender where to start sending more

fast retransmit

if large window + data packet 3 is lost, then sender will see

ACK 0, ACK 1, ACK 1, ACK 1, ACK 1, ACK 1

duplicate ACKs indicate missing packet

shouldn't wait for timeout

fast retransmit

if large window + data packet 3 is lost, then sender will see

ACK 0, ACK 1, ACK 1, ACK 1, ACK 1, ACK 1

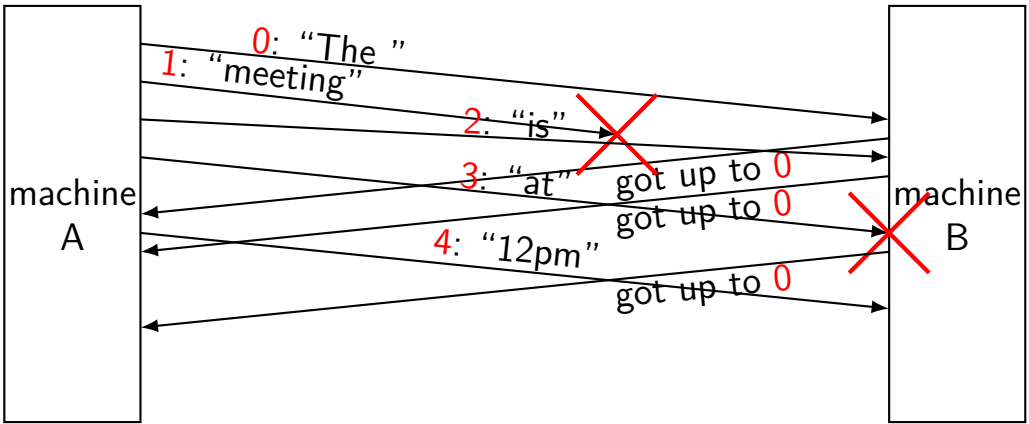
duplicate ACKs indicate missing packet

shouldn't wait for timeout

→ TCP heuristic: retransmit immediately after ∼3 duplicate ACKs

not 1 duplicate ACK to tolerate some reordering

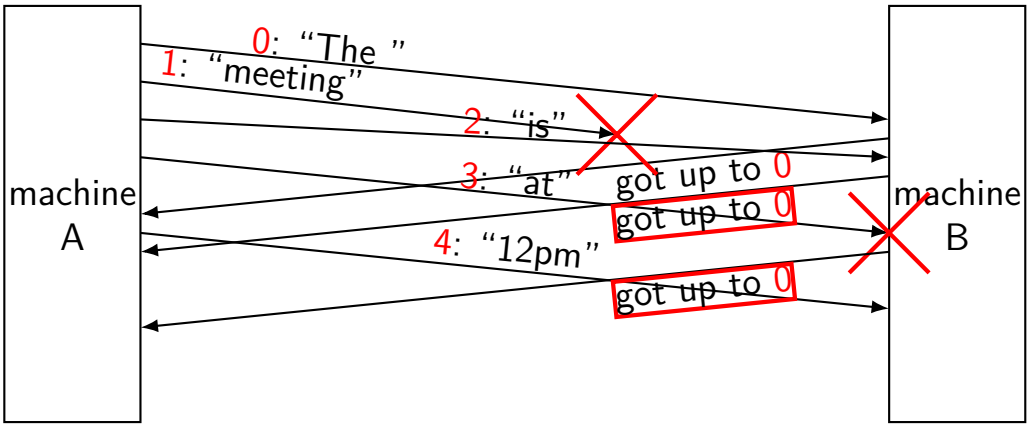
multiple missing



duplicate ACK heuristic will quickly resend 1, but not 3

would like to supply better information

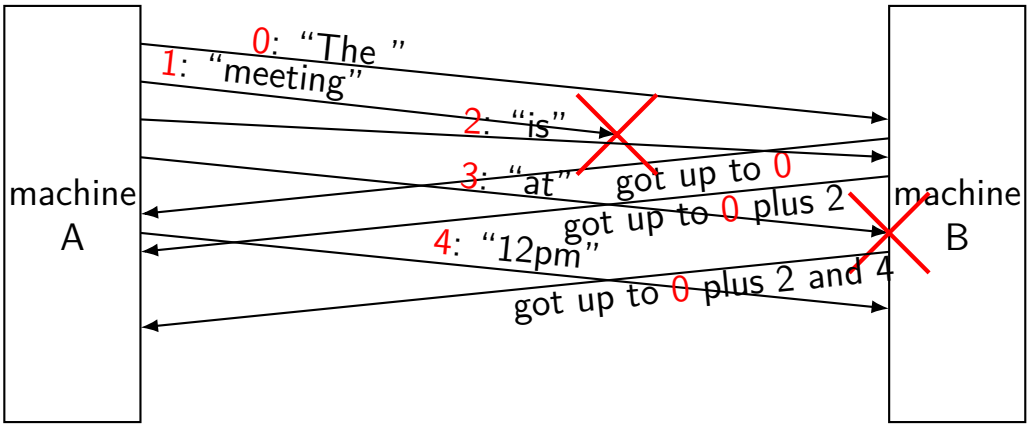
multiple missing



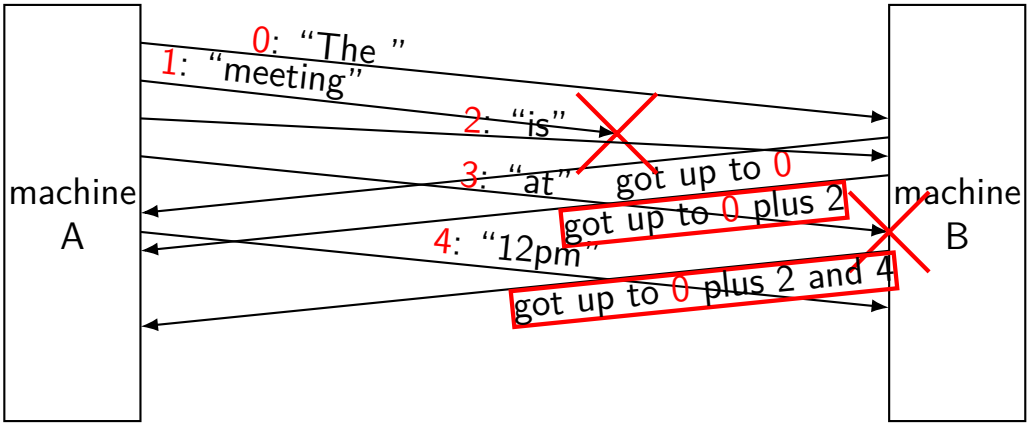
duplicate ACK heuristic will quickly resend 1, but not 3

would like to supply better information

selective acknowledgments



selective acknowledgments



selective acknowledgments in TCP

optional feature ("extension") described in RFC 2018

send list of ranges received

typically room for 3 ranges

if more than 3 ranges to report, then:

- include range with most recently received frame
- include other ranges until sent three times

backup slides